(一）基础题目3道题 50分

1.利用python的两种循环打印九九加法表（选择两种for-for,或 while-while进行）； 10分

```
In [ ]: for i in range(1, 10):
            for j in range(1, i + 1):
                result = i + j
                print(f"{j}+{i}={result:2}", end="  ")
            print()

        print("\n")
```

```
1+1= 2
1+2= 3  2+2= 4
1+3= 4  2+3= 5  3+3= 6
1+4= 5  2+4= 6  3+4= 7  4+4= 8
1+5= 6  2+5= 7  3+5= 8  4+5= 9  5+5=10
1+6= 7  2+6= 8  3+6= 9  4+6=10  5+6=11  6+6=12
1+7= 8  2+7= 9  3+7=10  4+7=11  5+7=12  6+7=13  7+7=14
1+8= 9  2+8=10  3+8=11  4+8=12  5+8=13  6+8=14  7+8=15  8+8=16
1+9=10  2+9=11  3+9=12  4+9=13  5+9=14  6+9=15  7+9=16  8+9=17  9+9=18
```

```
In [ ]: i = 1
        while i <= 9:
            j = 1
            while j <= i:
                result = i + j
                print(f"{j}+{i}={result:2}", end="  ")
                j += 1
            print()
            i += 1
```

```
1+1= 2
1+2= 3  2+2= 4
1+3= 4  2+3= 5  3+3= 6
1+4= 5  2+4= 6  3+4= 7  4+4= 8
1+5= 6  2+5= 7  3+5= 8  4+5= 9  5+5=10
1+6= 7  2+6= 8  3+6= 9  4+6=10  5+6=11  6+6=12
1+7= 8  2+7= 9  3+7=10  4+7=11  5+7=12  6+7=13  7+7=14
1+8= 9  2+8=10  3+8=11  4+8=12  5+8=13  6+8=14  7+8=15  8+8=16
1+9=10  2+9=11  3+9=12  4+9=13  5+9=14  6+9=15  7+9=16  8+9=17  9+9=18
```

2.个人所得税计算程序

需求说明： 编写一个Python程序，根据中国个人所得税税率表计算个人月所得税。程序应实现以下功能： 接收用户输入的月收入金额 计算应纳税所得额（月收入 - 起征点5000元 - 专项附加扣除1000元） 根据累进税率计算应缴个人所得税 请写出python代码。 10分

级数	预扣率（%）	速算扣除数
1	3	0
2	10	210
3	20	1410
4	25	2660
5	30	4410
6	35	7160
7	45	15160

```
In [ ]: def calculate_personal_income_tax(monthly_income):

            # 起征点和专项附加扣除
            threshold = 5000  # 起征点
            special_deduction = 1000  # 专项附加扣除

            # 计算应纳税所得额
            taxable_income = monthly_income - threshold - special_deduction

            # 如果应纳税所得额小于等于0，无需缴税
            if taxable_income <= 0:
                return taxable_income, 0

            # 税率表（应纳税所得额区间，预扣率，速算扣除数）
            tax_brackets = [
                (3000, 0.03, 0),       # 不超过3000元
                (12000, 0.10, 210),    # 超过3000元至12000元
                (25000, 0.20, 1410),   # 超过12000元至25000元
                (35000, 0.25, 2660),   # 超过25000元至35000元
                (55000, 0.30, 4410),   # 超过35000元至55000元
                (80000, 0.35, 7160),   # 超过55000元至80000元
                (float('inf'), 0.45, 15160)  # 超过80000元
            ]

            # 计算应缴税额
            for upper_limit, rate, deduction in tax_brackets:
                if taxable_income <= upper_limit:
                    tax = taxable_income * rate - deduction
                    return taxable_income, max(0, tax)

            return taxable_income, 0
```

```
In [ ]: def tax():

            try:
                monthly_income = float(input("请输入您的月收入金额（元）："))

                # 计算税额
                taxable_income, tax = calculate_personal_income_tax(monthly_income)

                # 输出结果
                print("计算结果：")
                print(f"月收入：{monthly_income:,.2f} 元")
                print(f"起征点：5,000.00 元")
                print(f"专项附加扣除：1,000.00 元")
                print(f"应纳税所得额：{taxable_income:,.2f} 元")
                print(f"应缴个人所得税：{tax:,.2f} 元")
                print(f"税后收入：{monthly_income - tax:,.2f} 元")

                if taxable_income > 0:
                    tax_rate = (tax / monthly_income) * 100
                    print(f"实际税率：{tax_rate:.2f}%")

            except ValueError:
                print("输入错误，请输入有效的数字！")
            except Exception as e:
                print(f"程序出错：{e}")
```

```
In [ ]: tax()
```

```
==================================================
个人所得税计算程序
==================================================
计算结果：
月收入：14,514.00 元
起征点：5,000.00 元
专项附加扣除：1,000.00 元
应纳税所得额：8,514.00 元
应缴个人所得税：641.40 元
税后收入：13,872.60 元
实际税率：4.42%
```

3.从tushare或者其他平台导入沪深300指数或者或者中证500指数，并从中挑选或者随机选择5只股票：

（1）进行合并，并存入本地（学号+姓名.csv格式）（5分）； （2）读取（1）存入本地的csv格式数据(学号+姓名.csv格式)，画出5只股票与沪深300指数或者5只股票与中证500指数的可比价格走势图（10分）； （3）画出这5只股票的有效前沿（15分）； 合计： 30分

```
In [ ]: import tushare as ts
        import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        import matplotlib.dates as mdates
        from datetime import datetime, timedelta
        from scipy.optimize import minimize
        import warnings
        warnings.filterwarnings('ignore')


        plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS']
        plt.rcParams['axes.unicode_minus'] = False


        ts.set_token('a394376ad4ede9c1214c06479e7d1bee32919045f909fcbfc39cabde')
        pro = ts.pro_api()

        print("开始使用tushare获取股票数据...")
```

```
开始使用tushare获取股票数据...
```

1.获取沪深300成分股并随机选择5只股票，获取数据并保存

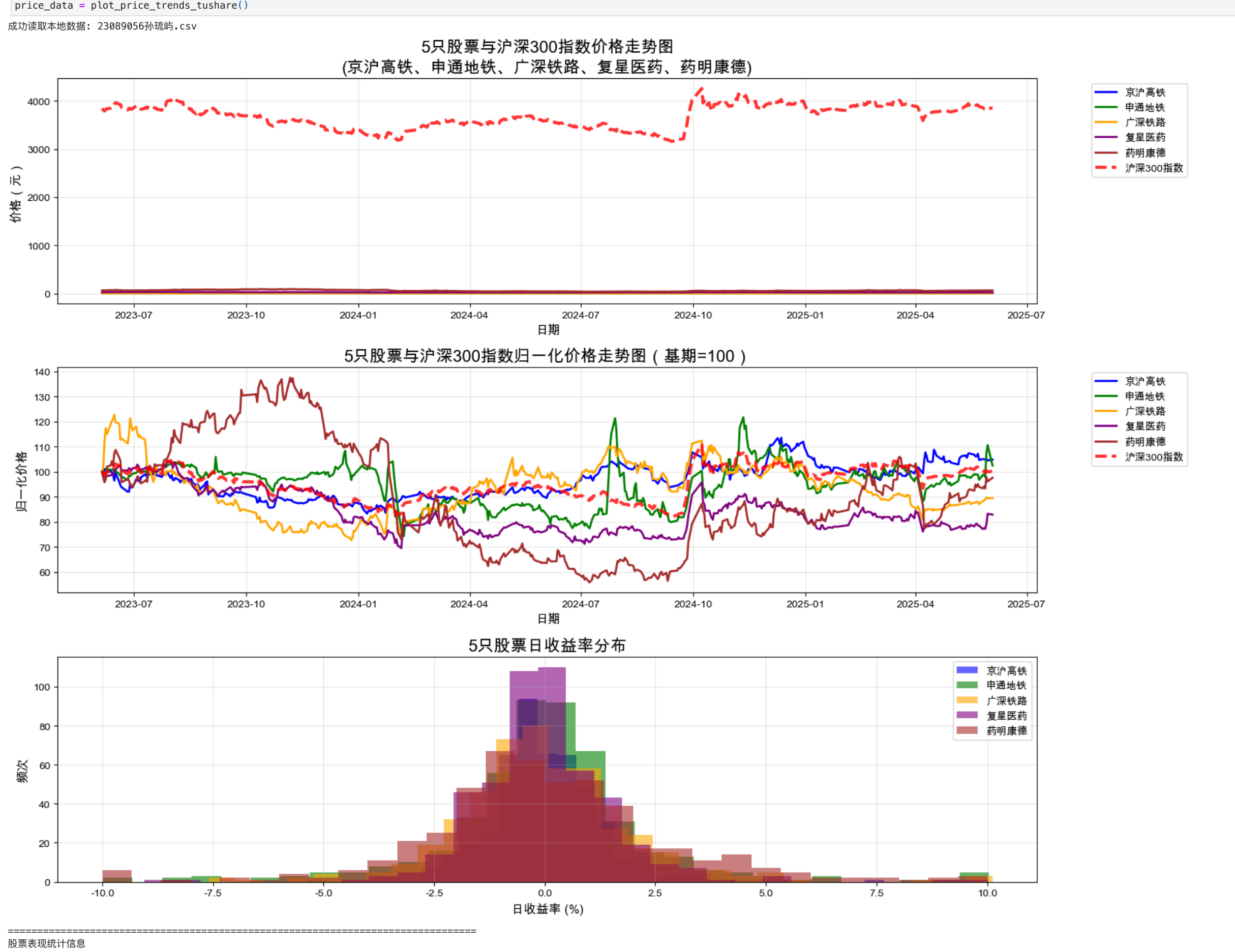
```
In [ ]: def get_stock_data_tushare():
            try:
                # 指定的5只股票
                selected_stocks = {
                    '601816.SH': '京沪高铁',
                    '600834.SH': '申通地铁',
                    '601333.SH': '广深铁路',
                    '600196.SH': '复星医药',
                    '603259.SH': '药明康德'
                }

                print("选择的5只股票:")
                for code, name in selected_stocks.items():
                    print(f"{code}: {name}")

                # 设置时间范围（最近2年）
                end_date = datetime.now().strftime('%Y%m%d')
                start_date = (datetime.now() - timedelta(days=730)).strftime('%Y%m%d')

                all_data = {}

                # 获取沪深300指数数据
                print("\n获取沪深300指数数据...")
                index_data = pro.index_daily(ts_code='399300.SZ', start_date=start_date, end_date=end_date)
                index_data['trade_date'] = pd.to_datetime(index_data['trade_date'])
                index_data = index_data.sort_values('trade_date').set_index('trade_date')
                all_data['沪深300指数'] = index_data['close']

                # 获取选中股票的数据
                for ts_code, stock_name in selected_stocks.items():
                    try:
                        print(f"获取{stock_name}({ts_code})数据...")
                        stock_data = pro.daily(ts_code=ts_code, start_date=start_date, end_date=end_date)

                        if not stock_data.empty:
                            stock_data['trade_date'] = pd.to_datetime(stock_data['trade_date'])
                            stock_data = stock_data.sort_values('trade_date').set_index('trade_date')
                            all_data[stock_name] = stock_data['close']
                        else:
                            print(f"警告: {stock_name}({ts_code})没有获取到数据")

                    except Exception as e:
                        print(f"获取{stock_name}数据失败: {e}")

                # 合并数据
                combined_data = pd.DataFrame(all_data)

                # 删除包含NaN的行
                combined_data = combined_data.dropna()

                # 保存到本地
                filename = '23089056孙琉屿.csv'
                combined_data.to_csv(filename, encoding='utf-8-sig')
                print(f"\n数据已保存到 {filename}")

                print(f"数据形状: {combined_data.shape}")
                print(f"数据时间范围: {combined_data.index[0]} 到 {combined_data.index[-1]}")
                print("\n数据预览:")
                print(combined_data.head())

                # 显示股票基本信息
                print("\n股票基本信息:")
                for code, name in selected_stocks.items():
                    try:
                        basic_info = pro.stock_basic(ts_code=code, fields='ts_code,symbol,name,area,industry,list_date')
                        if not basic_info.empty:
                            info = basic_info.iloc[0]
                            print(f"{name}({code}): 行业-{info['industry']}, 地区-{info['area']}, 上市日期-{info['list_date']}")
                    except:
                        print(f"{name}({code}): 基本信息获取失败")

                return combined_data, selected_stocks

            except Exception as e:
                print(f"数据获取失败: {e}")
                return None, None

        # 执行数据获取
        stock_data, selected_stocks_info = get_stock_data_tushare()
```

```
选择的5只股票:
601816.SH: 京沪高铁
600834.SH: 申通地铁
601333.SH: 广深铁路
600196.SH: 复星医药
603259.SH: 药明康德

获取沪深300指数数据...
获取京沪高铁(601816.SH)数据...
获取申通地铁(600834.SH)数据...
获取广深铁路(601333.SH)数据...
获取复星医药(600196.SH)数据...
获取药明康德(603259.SH)数据...

数据已保存到 23089056孙琉屿.csv
数据形状: (482, 6)
数据时间范围: 2023-06-05 00:00:00 到 2025-06-03 00:00:00

数据预览:
              沪深300指数  京沪高铁  申通地铁  广深铁路   复星医药   药明康德
trade_date
2023-06-05  3844.2492  5.61  8.30  3.40  30.50  66.09
2023-06-06  3808.1629  5.53  8.16  3.46  29.86  64.45
2023-06-07  3789.3418  5.67  8.26  3.81  29.83  63.28
2023-06-08  3820.1867  5.65  8.34  3.99  29.88  64.11
2023-06-09  3836.7026  5.64  8.38  3.91  29.95  67.44

股票基本信息:
京沪高铁(601816.SH): 行业-铁路, 地区-北京, 上市日期-20200116
申通地铁(600834.SH): 行业-公共交通, 地区-上海, 上市日期-19940224
广深铁路(601333.SH): 行业-铁路, 地区-深圳, 上市日期-20061222
复星医药(600196.SH): 行业-化学制药, 地区-上海, 上市日期-19980807
药明康德(603259.SH): 行业-化学制药, 地区-江苏, 上市日期-20180508
```

2.读取本地数据并画出价格走势图

```
In [ ]: def plot_price_trends_tushare():
            # 读取本地保存的数据
            filename = '23089056孙琉屿.csv'
            try:
                data = pd.read_csv(filename, index_col=0, parse_dates=True, encoding='utf-8-sig')
                print(f"成功读取本地数据: {filename}")
            except FileNotFoundError:
                print(f"未找到文件 {filename}，请先运行数据获取程序")
                return None


            normalized_data = (data / data.iloc[0]) * 100


            plt.figure(figsize=(16, 12))


            stock_columns = [col for col in data.columns if col != '沪深300指数']


            plt.subplot(3, 1, 1)
            colors = ['blue', 'green', 'orange', 'purple', 'brown']  # 为5只股票指定颜色

            for i, stock in enumerate(stock_columns):
                plt.plot(data.index, data[stock], label=stock, linewidth=2, color=colors[i])
            plt.plot(data.index, data['沪深300指数'], label='沪深300指数',
                    linewidth=3, color='red', linestyle='--', alpha=0.8)

            plt.title('5只股票与沪深300指数价格走势图\n(京沪高铁、申通地铁、广深铁路、复星医药、药明康德)',
                      fontsize=16, fontweight='bold')
            plt.xlabel('日期', fontsize=12)
            plt.ylabel('价格（元）', fontsize=12)
            plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
            plt.grid(True, alpha=0.3)


            plt.subplot(3, 1, 2)
            for i, stock in enumerate(stock_columns):
                plt.plot(normalized_data.index, normalized_data[stock],
                        label=stock, linewidth=2, color=colors[i])
            plt.plot(normalized_data.index, normalized_data['沪深300指数'],
                    label='沪深300指数', linewidth=3, color='red', linestyle='--', alpha=0.8)

            plt.title('5只股票与沪深300指数归一化价格走势图（基期=100）', fontsize=16, fontweight='bold')
            plt.xlabel('日期', fontsize=12)
            plt.ylabel('归一化价格', fontsize=12)
            plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
            plt.grid(True, alpha=0.3)


            plt.subplot(3, 1, 3)
            returns = data.pct_change().dropna()

            for i, stock in enumerate(stock_columns):
                plt.hist(returns[stock] * 100, alpha=0.6, bins=30,
                        label=f'{stock}', color=colors[i])

            plt.title('5只股票日收益率分布', fontsize=16, fontweight='bold')
            plt.xlabel('日收益率 (%)', fontsize=12)
            plt.ylabel('频次', fontsize=12)
            plt.legend()
            plt.grid(True, alpha=0.3)

            plt.tight_layout()
            plt.show()

            # 计算并显示统计信息
            print("=" * 80)
            print("股票表现统计信息")
            print("=" * 80)

            returns = data.pct_change().dropna()

            stats = pd.DataFrame({
                '期初价格': data.iloc[0],
                '期末价格': data.iloc[-1],
                '累计收益率(%)': ((data.iloc[-1] / data.iloc[0] - 1) * 100),
                '年化收益率(%)': ((data.iloc[-1] / data.iloc[0]) ** (252/len(data)) - 1) * 100,
                '年化波动率(%)': (returns.std() * np.sqrt(252) * 100),
                '最大回撤(%)': ((data / data.expanding().max() - 1).min() * 100),
                '夏普比率': (returns.mean() * 252) / (returns.std() * np.sqrt(252))
            })

            print(stats.round(2))

            # 相关性分析
            print("\n" + "=" * 60)
            print("股票与沪深300指数的相关性")
            print("=" * 60)
            correlation_matrix = returns.corr()
            print(correlation_matrix.round(3))

            # 行业分析
            print("\n" + "=" * 60)
            print("行业分析")
            print("=" * 60)
            print("交通运输板块: 京沪高铁、申通地铁、广深铁路")
            print("医药板块: 复星医药、药明康德")

            transport_stocks = ['京沪高铁', '申通地铁', '广深铁路']
            pharma_stocks = ['复星医药', '药明康德']

            transport_avg_return = returns[transport_stocks].mean(axis=1)
            pharma_avg_return = returns[pharma_stocks].mean(axis=1)

            print(f"交通运输板块平均年化收益率: {transport_avg_return.mean() * 252:.2%}")
            print(f"医药板块平均年化收益率: {pharma_avg_return.mean() * 252:.2%}")
            print(f"交通运输板块平均年化波动率: {transport_avg_return.std() * np.sqrt(252):.2%}")
            print(f"医药板块平均年化波动率: {pharma_avg_return.std() * np.sqrt(252):.2%}")

            return data

        # 执行价格走势图绘制
        price_data = plot_price_trends_tushare()
```

```
成功读取本地数据: 23089056孙琉屿.csv
```

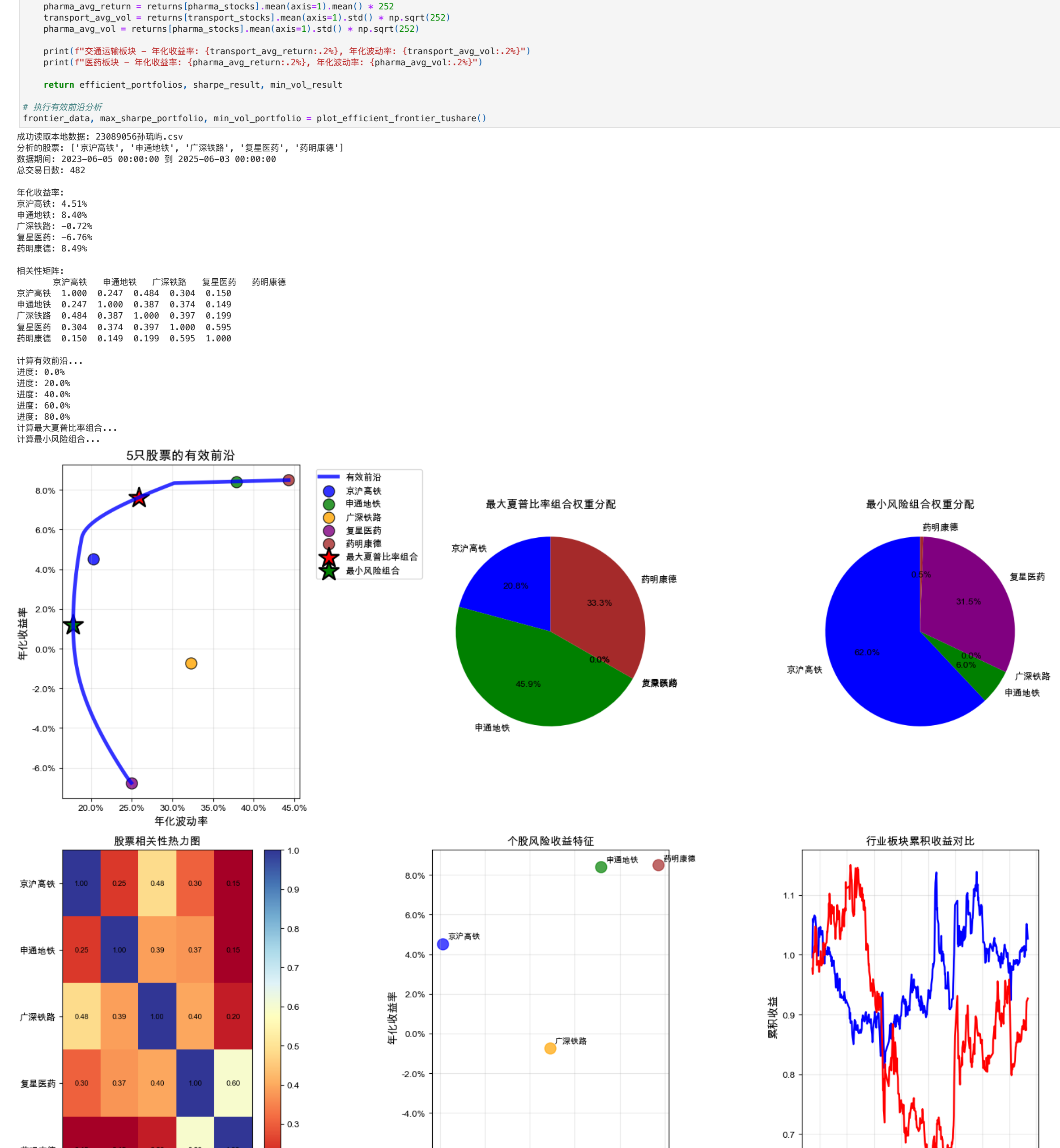


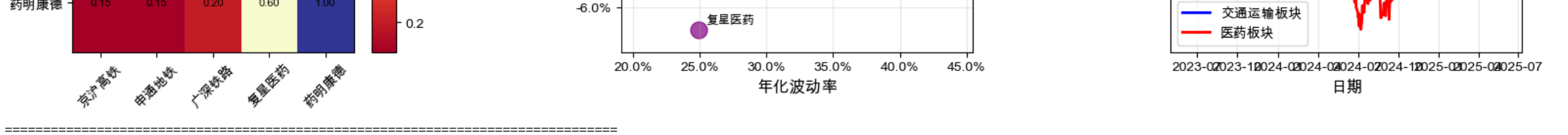
```
================================================================================
股票表现统计信息
================================================================================
           期初价格     期末价格  累计收益率(%)  年化收益率(%)  年化波动率(%)  最大回撤(%)  夏普比率
沪深300指数  3844.25  3852.01      0.20      0.11     18.55   -21.42  0.10
京沪高铁       5.61     5.88      4.81      2.49     20.30   -17.64  0.22
申通地铁       8.30     8.50      2.41      1.25     37.89   -34.16  0.22
广深铁路       3.40     3.04    -10.59     -5.68     32.24   -40.77 -0.02
复星医药      30.50    25.27    -17.15     -9.37     24.93   -33.20 -0.27
药明康德      66.09    64.47     -2.45     -1.29     44.29   -59.43  0.19

============================================================
股票与沪深300指数的相关性
============================================================
         沪深300指数   京沪高铁   申通地铁   广深铁路   复星医药   药明康德
沪深300指数    1.000  0.485  0.362  0.486  0.740  0.540
京沪高铁     0.485  1.000  0.247  0.484  0.304  0.150
申通地铁     0.362  0.247  1.000  0.387  0.374  0.149
广深铁路     0.486  0.484  0.387  1.000  0.397  0.199
复星医药     0.740  0.304  0.374  0.397  1.000  0.595
药明康德     0.540  0.150  0.149  0.199  0.595  1.000

============================================================
行业分析
============================================================
交通运输板块: 京沪高铁、申通地铁、广深铁路
医药板块: 复星医药、药明康德
交通运输板块平均年化收益率: 4.06%
医药板块平均年化收益率: 0.87%
交通运输板块平均年化波动率: 23.21%
医药板块平均年化波动率: 31.22%
```

3.计算并绘制5只股票的有效前沿

```
In [ ]: def plot_efficient_frontier_tushare():
            # 读取数据
            filename = '23089056孙琉屿.csv'
            try:
                data = pd.read_csv(filename, index_col=0, parse_dates=True, encoding='utf-8-sig')
                print(f"成功读取本地数据: {filename}")
            except FileNotFoundError:
                print(f"未找到文件 {filename}，请先运行数据获取程序")
                return None

            # 只选择5只股票（排除指数）
            stock_columns = [col for col in data.columns if col != '沪深300指数']
            stock_data = data[stock_columns]

            print(f"分析的股票: {list(stock_columns)}")
            print(f"数据期间: {stock_data.index[0]} 到 {stock_data.index[-1]}")
            print(f"总交易日数: {len(stock_data)}")

            # 计算日收益率
            returns = stock_data.pct_change().dropna()

            # 计算年化收益率和协方差矩阵
            mean_returns = returns.mean() * 252  # 年化
            cov_matrix = returns.cov() * 252     # 年化

            print("\n年化收益率:")
            for stock in stock_columns:
                print(f"{stock}: {mean_returns[stock]:.2%}")

            print("\n相关性矩阵:")
            corr_matrix = returns.corr()
            print(corr_matrix.round(3))

            # 定义投资组合优化函数
            def portfolio_stats(weights, mean_returns, cov_matrix):
                portfolio_return = np.sum(mean_returns * weights)
                portfolio_volatility = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights)))
                return portfolio_return, portfolio_volatility

            def negative_sharpe(weights, mean_returns, cov_matrix, risk_free_rate=0.03):
                p_ret, p_vol = portfolio_stats(weights, mean_returns, cov_matrix)
                return -(p_ret - risk_free_rate) / p_vol

            def portfolio_volatility(weights, mean_returns, cov_matrix):
                return portfolio_stats(weights, mean_returns, cov_matrix)[1]

            # 约束条件和边界
            num_assets = len(stock_columns)
            constraints = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1})
            bounds = tuple((0, 1) for _ in range(num_assets))
            initial_guess = num_assets * [1/num_assets]

            # 计算有效前沿
            target_returns = np.linspace(mean_returns.min(), mean_returns.max(), 100)
            efficient_portfolios = []

            print("\n计算有效前沿...")
            for i, target in enumerate(target_returns):
                if i % 20 == 0:
                    print(f"进度: {i/len(target_returns)*100:.1f}%")

                constraints_with_return = [
                    {'type': 'eq', 'fun': lambda x: np.sum(x) - 1},
                    {'type': 'eq', 'fun': lambda x: np.sum(mean_returns * x) - target}
                ]

                result = minimize(portfolio_volatility,
                                 initial_guess,
                                 args=(mean_returns, cov_matrix),
                                 method='SLSQP',
                                 bounds=bounds,
                                 constraints=constraints_with_return,
                                 options={'ftol': 1e-9, 'disp': False})

                if result.success:
                    efficient_portfolios.append([target, result.fun])

            efficient_portfolios = np.array(efficient_portfolios)

            # 计算最大夏普比率组合
            print("计算最大夏普比率组合...")
            sharpe_result = minimize(negative_sharpe,
                                   initial_guess,
                                   args=(mean_returns, cov_matrix),
                                   method='SLSQP',
                                   bounds=bounds,
                                   constraints=constraints)

            max_sharpe_return, max_sharpe_volatility = portfolio_stats(sharpe_result.x, mean_returns, cov_matrix)

            # 计算最小风险组合
            print("计算最小风险组合...")
            min_vol_result = minimize(portfolio_volatility,
                                    initial_guess,
                                    args=(mean_returns, cov_matrix),
                                    method='SLSQP',
                                    bounds=bounds,
                                    constraints=constraints)

            min_vol_return, min_vol_volatility = portfolio_stats(min_vol_result.x, mean_returns, cov_matrix)

            # 绘制有效前沿
            plt.figure(figsize=(16, 12))

            # 主图：有效前沿
            plt.subplot(2, 3, 1)

            # 绘制有效前沿
            plt.plot(efficient_portfolios[:, 1], efficient_portfolios[:, 0],
                    'b-', linewidth=4, label='有效前沿', alpha=0.8)

            # 绘制个股
            colors = ['blue', 'green', 'orange', 'purple', 'brown']
            for i, stock in enumerate(stock_columns):
                plt.scatter(np.sqrt(cov_matrix.iloc[i, i]), mean_returns.iloc[i],
                          marker='o', s=150, color=colors[i], label=stock,
                          edgecolors='black', linewidth=1, alpha=0.8)

            # 标记特殊点
            plt.scatter(max_sharpe_volatility, max_sharpe_return,
                       marker='*', s=500, color='red', label='最大夏普比率组合',
                       edgecolors='black', linewidth=2)

            plt.scatter(min_vol_volatility, min_vol_return,
                       marker='*', s=500, color='green', label='最小风险组合',
                       edgecolors='black', linewidth=2)

            plt.xlabel('年化波动率', fontsize=12)
            plt.ylabel('年化收益率', fontsize=12)
            plt.title('5只股票的有效前沿', fontsize=14, fontweight='bold')
            plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
            plt.grid(True, alpha=0.3)

            # 格式化坐标轴为百分比
            plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: '{:.1%}'.format(y)))
            plt.gca().xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: '{:.1%}'.format(x)))

            # 子图2：最大夏普比率组合权重
            plt.subplot(2, 3, 2)
            plt.pie(sharpe_result.x, labels=stock_columns, autopct='%1.1f%%',
                   startangle=90, colors=colors)
            plt.title('最大夏普比率组合权重分配', fontsize=12, fontweight='bold')

            # 子图3：最小风险组合权重
            plt.subplot(2, 3, 3)
            plt.pie(min_vol_result.x, labels=stock_columns, autopct='%1.1f%%',
                   startangle=90, colors=colors)
            plt.title('最小风险组合权重分配', fontsize=12, fontweight='bold')

            # 子图4：相关性热力图
            plt.subplot(2, 3, 4)
            corr_matrix = returns.corr()
            im = plt.imshow(corr_matrix, cmap='RdYlBu', aspect='auto')
            plt.colorbar(im)
            plt.xticks(range(len(stock_columns)), stock_columns, rotation=45)
            plt.yticks(range(len(stock_columns)), stock_columns)
            plt.title('股票相关性热力图', fontsize=12, fontweight='bold')

            # 在每个格子中显示相关性数值
            for i in range(len(stock_columns)):
                for j in range(len(stock_columns)):
                    plt.text(j, i, f'{corr_matrix.iloc[i,j]:.2f}',
                            ha='center', va='center', fontsize=8)

            # 子图5：风险收益散点图
            plt.subplot(2, 3, 5)
            for i, stock in enumerate(stock_columns):
                plt.scatter(np.sqrt(cov_matrix.iloc[i, i]), mean_returns.iloc[i],
                          s=150, color=colors[i], alpha=0.7)
                plt.annotate(stock, (np.sqrt(cov_matrix.iloc[i, i]), mean_returns.iloc[i]),
                            xytext=(5, 5), textcoords='offset points', fontsize=9)

            plt.xlabel('年化波动率', fontsize=12)
            plt.ylabel('年化收益率', fontsize=12)
            plt.title('个股风险收益特征', fontsize=12, fontweight='bold')
            plt.grid(True, alpha=0.3)
            plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: '{:.1%}'.format(y)))
            plt.gca().xaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: '{:.1%}'.format(x)))

            # 子图6：行业对比
            plt.subplot(2, 3, 6)
            transport_returns = returns[['京沪高铁', '申通地铁', '广深铁路']].mean(axis=1)
            pharma_returns = returns[['复星医药', '药明康德']].mean(axis=1)

            cumulative_transport = (1 + transport_returns).cumprod()
            cumulative_pharma = (1 + pharma_returns).cumprod()

            plt.plot(cumulative_transport.index, cumulative_transport,
                    label='交通运输板块', linewidth=2, color='blue')
            plt.plot(cumulative_pharma.index, cumulative_pharma,
                    label='医药板块', linewidth=2, color='red')

            plt.xlabel('日期', fontsize=12)
            plt.ylabel('累积收益', fontsize=12)
            plt.title('行业板块累积收益对比', fontsize=12, fontweight='bold')
            plt.legend()
            plt.grid(True, alpha=0.3)

            plt.tight_layout()
            plt.show()

            # 输出详细分析结果
            print("\n" + "=" * 80)
            print("投资组合优化分析结果")
            print("=" * 80)

            print(f"\n【最大夏普比率组合】")
            print(f"预期年化收益率: {max_sharpe_return:.2%}")
            print(f"年化波动率: {max_sharpe_volatility:.2%}")
            print(f"夏普比率: {(max_sharpe_return - 0.03) / max_sharpe_volatility:.3f}")
            print("权重分配:")
            for i, stock in enumerate(stock_columns):
                print(f"  {stock}: {sharpe_result.x[i]:.1%}")

            print(f"\n【最小风险组合】")
            print(f"预期年化收益率: {min_vol_return:.2%}")
            print(f"年化波动率: {min_vol_volatility:.2%}")
            print("权重分配:")
            for i, stock in enumerate(stock_columns):
                print(f"  {min_vol_result.x[i]:.1%}")

            print(f"\n【等权重组合（基准）】")
            equal_weights = np.array([1/num_assets] * num_assets)
            equal_return, equal_vol = portfolio_stats(equal_weights, mean_returns, cov_matrix)
            equal_sharpe = (equal_return - 0.03) / equal_vol
            print(f"预期年化收益率: {equal_return:.2%}")
            print(f"年化波动率: {equal_vol:.2%}")
            print(f"夏普比率: {equal_sharpe:.3f}")

            print(f"\n【行业分析】")
            print("交通运输板块 vs 医药板块表现对比:")
            transport_stocks = ['京沪高铁', '申通地铁', '广深铁路']
            pharma_stocks = ['复星医药', '药明康德']

            transport_avg_return = returns[transport_stocks].mean(axis=1).mean() * 252
            pharma_avg_return = returns[pharma_stocks].mean(axis=1).mean() * 252
            transport_avg_vol = returns[transport_stocks].mean(axis=1).std() * np.sqrt(252)
            pharma_avg_vol = returns[pharma_stocks].mean(axis=1).std() * np.sqrt(252)

            print(f"交通运输板块 - 年化收益率: {transport_avg_return:.2%}, 年化波动率: {transport_avg_vol:.2%}")
            print(f"医药板块 - 年化收益率: {pharma_avg_return:.2%}, 年化波动率: {pharma_avg_vol:.2%}")

            return efficient_portfolios, sharpe_result, min_vol_result

        # 执行有效前沿分析
        frontier_data, max_sharpe_portfolio, min_vol_portfolio = plot_efficient_frontier_tushare()
```

```
成功读取本地数据: 23089056孙琉屿.csv
分析的股票: ['京沪高铁', '申通地铁', '广深铁路', '复星医药', '药明康德']
数据期间: 2023-06-05 00:00:00 到 2025-06-03 00:00:00
总交易日数: 482

年化收益率:
京沪高铁: 4.51%
申通地铁: 8.40%
广深铁路: -0.72%
复星医药: -6.76%
药明康德: 8.49%

相关性矩阵:
        京沪高铁   申通地铁   广深铁路   复星医药   药明康德
京沪高铁  1.000  0.247  0.484  0.304  0.150
申通地铁  0.247  1.000  0.387  0.374  0.149
广深铁路  0.484  0.387  1.000  0.397  0.199
复星医药  0.304  0.374  0.397  1.000  0.595
药明康德  0.150  0.149  0.199  0.595  1.000

计算有效前沿...
进度: 0.0%
进度: 20.0%
进度: 40.0%
进度: 60.0%
进度: 80.0%
计算最大夏普比率组合...
计算最小风险组合...
```





```
================================================================================
投资组合优化分析结果
================================================================================

【最大夏普比率组合】
预期年化收益率: 7.62%
年化波动率: 25.87%
夏普比率: 0.179
权重分配:
  京沪高铁: 20.8%
  申通地铁: 45.9%
  广深铁路: 0.0%
  复星医药: 0.0%
  药明康德: 33.3%

【最小风险组合】
预期年化收益率: 1.21%
年化波动率: 17.77%
权重分配:
  62.0%
  6.0%
  0.0%
  31.5%
  0.5%

【等权重组合（基准）】
预期年化收益率: 2.78%
年化波动率: 21.67%
夏普比率: -0.010

【行业分析】
交通运输板块 vs 医药板块表现对比:
交通运输板块 - 年化收益率: 4.06%, 年化波动率: 23.21%
医药板块 - 年化收益率: 0.87%, 年化波动率: 31.22%
```

（二）应用项目 50分

利用Python做一个项目，比如 （1）所有上市公司5年的资产负债表（选2-3个指标）、利润表（选2-3个指标）进行合并，计算相关财务指标；或者（2）一个金融数据统计分析与可视化；或者 （3）一个量化交易策略；或者 （4）一个爬虫案例等，都可以。 要求有：python代码和在jupyter中运行的最终结果。 50分

全球重大事件对世界金融市场影响分析

本代码主要用于分析全球重大事件对主要金融市场指数和国际金价的短期影响。通过数据收集、处理、分析和可视化，直观地展示了各类事件与金融工具表现之间的潜在关联。

1.使用的方法

(1).数据获取

· 代码使用 Tushare 库获取上证指数的自2010年至今每日收盘数据。

· 代码使用 yfinance 库获取纳斯达克指数、道琼斯指数以及国际金价自2010年至今每日收盘数据。

· 全球重大事件数据（包括自然灾害、经济事件、地缘政治事件、科技进展等）预先由本人在互联网上搜索并导入代码进行数据处理。

(2).数据处理与归一化

· 所有金融工具的原始价格数据都被归一化，使其从各自数据起始点的100点开始，便于在同一图表上进行趋势的相对比较，避免因绝对价格差异过大而导致的视觉偏差。

· 事件日期被转换为统一的日期时间格式，以便与金融数据进行匹配。

(3).事件影响分析

· 针对每个历史事件，代码计算了事件发生后 5个交易日内 各金融工具的百分比变化。这有助于评估事件对市场的短期影响。

· 代码还计算了不同事件类别（如自然灾害、经济事件等）对每种金融工具的 平均百分比变化，以量化各类事件的整体影响趋势。

(4).数据可视化

· 代码采用 Plotly 库生成交互式图表，增强了数据探索的灵活性。

2.实现功能

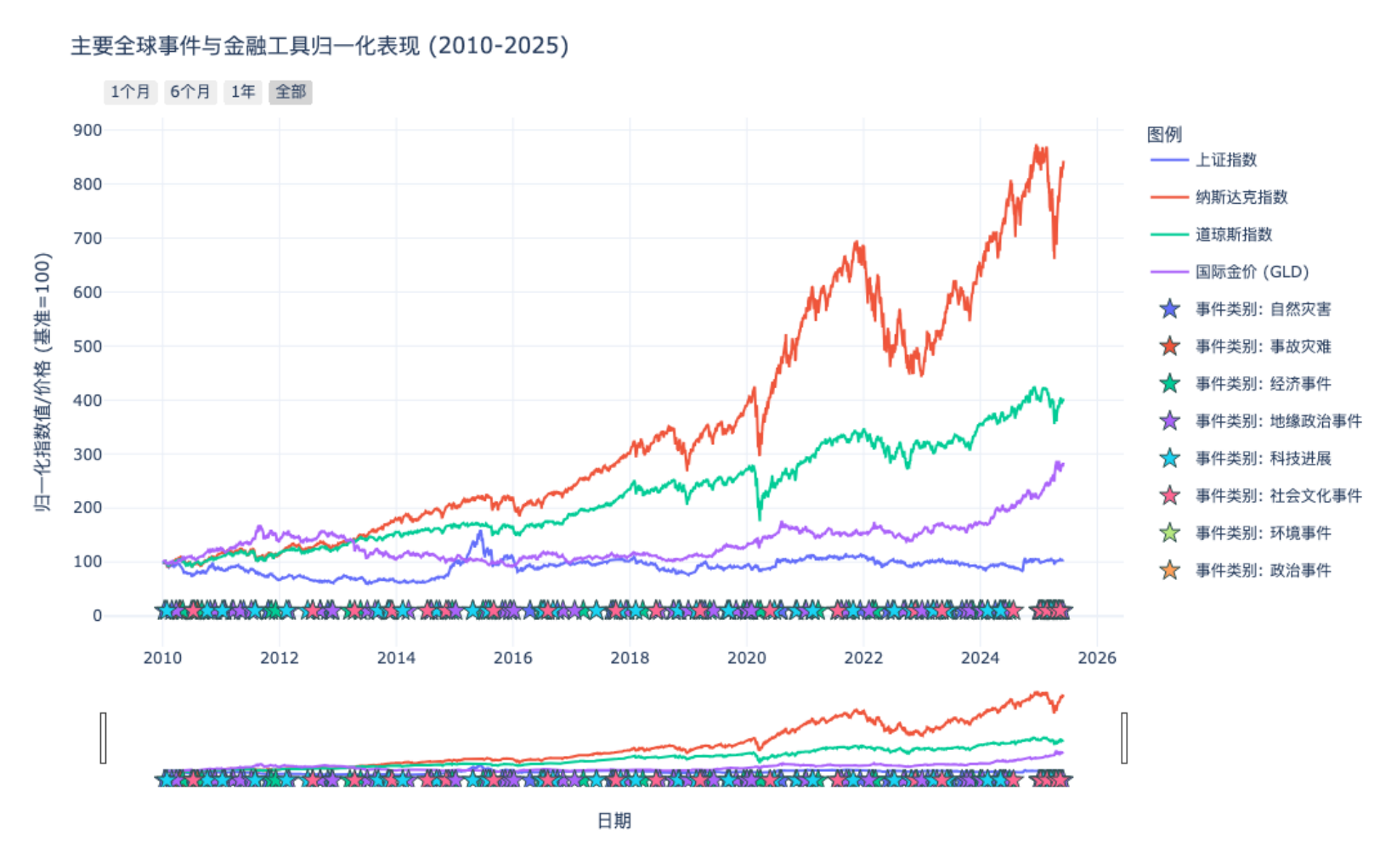
代码通过图表和数据表格，帮助您理解全球重大事件如何短期影响金融市场和黄金价格，并比较不同类型事件的平均影响。

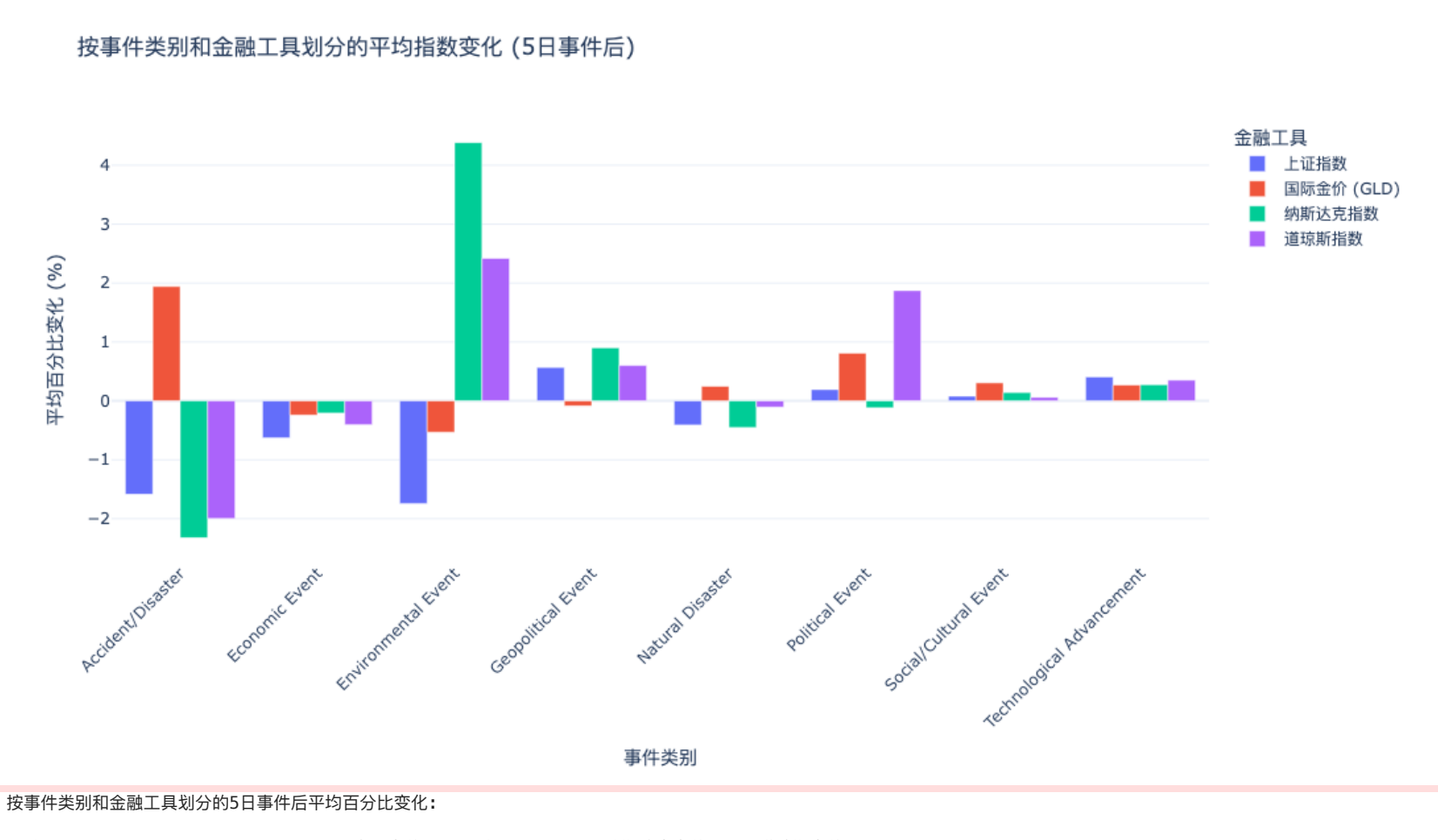
3.代码

In [3]:

```python
import tushare as ts
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
import numpy as np
import yfinance as yf # Import yfinance library

# Set your Tushare token here.
TS_TOKEN = 'a394376ad4ede9c1214c06479e7d1bee32919045f909fcbfc39cabde'
ts.set_token(TS_TOKEN)
pro = ts.pro_api()

# --- 1. Load Major Global Events Data ---
# The events data is extracted from the events.py file provided by the user.
# For this demonstration, the data is manually included in the script.

events_2010 = [
    ("2010-01-12", "海地发生7.0级地震，约30万人丧生", "自然灾害"),
    ("2010-02-27", "智利发生8.8级地震并引发海啸，造成500余人死亡", "自然灾害"),
    ("2010-04-14", "中国青海玉树发生7.1级地震，2698人遇难", "自然灾害"),
    ("2010-07-01", "巴基斯坦特大洪灾，约2000万人受灾", "自然灾害"),
    ("2010-08-08", "中国甘肃舟曲特大泥石流灾害，1501人遇难", "自然灾害"),
    ("2010-10-25", "印尼地震引发海啸及火山喷发，500余人死亡", "自然灾害"),

    ("2010-04-20", "墨西哥湾深水地平线钻井平台爆炸，引发美国史上最严重原油泄漏", "事故灾难"),
    ("2010-08-05", "智利圣何塞铜矿塌方，33名矿工被困69天后获救", "事故灾难"),
    ("2010-11-22", "柬埔寨金边钻石桥踩踏事件，353人死亡", "事故灾难"),

    ("2010-05-06", "美股闪电崩盘", "经济事件"),
    ("2010-07-21", "美国多德-弗兰克法案签署，加强金融监管", "经济事件"),
    ("2010-09-02", "爱尔兰银行危机恶化，寻求欧盟援助", "经济事件"),

    ("2010-03-26", "天安舰事件，韩朝关系紧张", "地缘政治事件"),
    ("2010-11-23", "延坪岛炮击事件，韩朝冲突升级", "地缘政治事件"),

    ("2010-01-27", "苹果公司发布iPad", "科技进展"),
    ("2010-10-06", "Instagram发布", "科技进展"),

    ("2010-07-11", "南非世界杯举行，首次在非洲举办", "社会文化事件"),
]

events_2011 = [
    ("2011-03-11", "日本9.0级地震及海啸，引发福岛核事故", "自然灾害"),
    ("2011-05-22", "美国密苏里州乔普林遭遇EF5级龙卷风，161人死亡", "自然灾害"),
    ("2011-10-23", "土耳其凡城地震，604人死亡", "自然灾害"),
    ("2011-10-23", "泰国特大洪灾，持续数月影响全球硬盘供应链", "自然灾害"),
    ("2011-08-05", "标普下调美国主权信用评级", "经济事件"),
    ("2011-11-11", "希腊债务危机深化，新政府成立", "经济事件"),
    ("2011-12-09", "欧元区峰会通过“财政契约”草案，加强财政纪律", "经济事件"),
    ("2011-01-25", "埃及革命爆发，穆巴拉克下台", "地缘政治事件"),
    ("2011-03-19", "利比亚战争开始，多国实施军事干预", "地缘政治事件"),
    ("2011-05-02", "本·拉登在巴基斯坦被击毙", "地缘政治事件"),
    ("2011-07-09", "南苏丹共和国独立", "地缘政治事件"),
    ("2011-01-20", "卡扎菲在利比亚被击毙", "地缘政治事件"),
    ("2011-01-09", "比特币首次突破1美元", "科技进展"),
    ("2011-08-01", "Google+发布", "科技进展"),
]

events_2012 = [
    ("2012-10-29", "飓风桑迪袭击美国东海岸，造成巨大损失", "自然灾害"),
    ("2012-07-26", "欧洲央行行长德拉吉承诺“不惜一切代价”拯救欧元", "经济事件"),
    ("2012-09-13", "美联储推出第三轮量化宽松（QE3）", "经济事件"),
    ("2012-11-20", "欧元区援助希腊达成协议", "经济事件"),
    ("2012-03-01", "联合国安理会未能通过叙利亚决议，叙利亚内战升级", "地缘政治事件"),
    ("2012-09-11", "美国驻利比亚班加西领事馆遇袭", "地缘政治事件"),
    ("2012-11-29", "巴勒斯坦获得联合国观察员国地位", "地缘政治事件"),
    ("2012-02-14", "树莓派发布", "科技进展"),
    ("2012-06-25", "Google I/O大会发布Google Glass", "科技进展"),
    ("2012-07-27", "伦敦奥运会开幕", "社会文化事件"),
]

events_2013 = [
    ("2013-11-08", "超强台风海燕袭击菲律宾，造成6300多人死亡", "自然灾害"),
    ("2013-03-16", "塞浦路斯银行危机，首次对银行存款征税", "经济事件"),
    ("2013-05-22", "美联储暗示可能缩减量化宽松，引发“缩减恐慌”", "经济事件"),
    ("2013-07-03", "埃及军事政变，穆尔西被罢黜", "地缘政治事件"),
    ("2013-08-21", "叙利亚化武袭击事件，国际社会强烈谴责", "地缘政治事件"),
    ("2013-11-24", "伊朗与P5+1达成核问题初步协议", "地缘政治事件"),
    ("2013-09-10", "苹果公司发布iPhone 5s和5c，引入Touch ID", "科技进展"),
    ("2013-11-15", "索尼发布PlayStation 4", "科技进展"),
    ("2013-04-15", "波士顿马拉松爆炸案", "社会文化事件"),
    ("2013-12-05", "南非前总统曼德拉逝世", "社会文化事件"),
]

events_2014 = [
    ("2014-08-03", "中国云南鲁甸6.5级地震，617人死亡", "自然灾害"),
    ("2014-09-27", "日本御岳山火山喷发", "自然灾害"),
    ("2014-12-16", "俄罗斯金融危机爆发，卢布大幅贬值", "经济事件"),
    ("2014-03-18", "克里米亚并入俄罗斯", "地缘政治事件"),
    ("2014-07-17", "马航MH17在乌克兰东部被击落", "地缘政治事件"),
    ("2014-08-08", "美国开始对伊拉克和叙利亚境内的ISIS进行空袭", "地缘政治事件"),
    ("2014-11-24", "美国与古巴恢复外交关系", "地缘政治事件"),
    ("2014-02-10", "Facebook收购WhatsApp", "科技进展"),
    ("2014-09-09", "苹果公司发布Apple Watch和iPhone 6", "科技进展"),
    ("2014-07-13", "德国赢得巴西世界杯冠军", "社会文化事件"),
    ("2014-11-09", "柏林墙倒塌25周年纪念", "社会文化事件"),
]

events_2015 = [
    ("2015-04-25", "尼泊尔8.1级地震，近9000人死亡", "自然灾害"),
    ("2015-10-26", "阿富汗7.5级地震，造成数百人死亡", "自然灾害"),
    ("2015-06-27", "希腊债务谈判破裂，面临退出欧元区风险", "经济事件"),
    ("2015-08-11", "中国央行完善人民币兑美元汇率中间价报价，人民币贬值", "经济事件"),
    ("2015-12-16", "美联储近十年来首次加息", "经济事件"),
    ("2015-01-07", "法国查理周刊袭击事件", "地缘政治事件"),
    ("2015-07-14", "伊朗核协议达成", "地缘政治事件"),
    ("2015-11-13", "巴黎系列恐怖袭击事件", "地缘政治事件"),
    ("2015-12-12", "《巴黎协定》通过", "地缘政治事件"),
    ("2015-04-24", "Apple Watch正式发售", "科技进展"),
    ("2015-07-29", "微软发布Windows 10", "科技进展"),
    ("2015-09-02", "艾兰·库尔迪（小难民）照片引发全球关注", "社会文化事件"),
]

events_2016 = [
    ("2016-04-16", "日本九州连续发生强震", "自然灾害"),
    ("2016-10-04", "飓风马修袭击海地、古巴和美国东南部", "自然灾害"),
    ("2016-06-23", "英国脱欧公投，决定退出欧盟", "经济事件"),
    ("2016-01-06", "朝鲜进行第四次核试验", "地缘政治事件"),
    ("2016-07-15", "土耳其军事政变未遂", "地缘政治事件"),
    ("2016-11-08", "唐纳德·特朗普当选美国总统", "地缘政治事件"),
    ("2016-07-06", "Pokemon Go发布", "科技进展"),
    ("2016-09-07", "苹果公司发布iPhone 7，取消耳机孔", "科技进展"),
    ("2016-08-05", "里约奥运会开幕", "社会文化事件"),
]

events_2017 = [
    ("2017-09-06", "飓风厄玛、哈维、玛丽亚相继袭击加勒比和美国", "自然灾害"),
    ("2017-09-19", "墨西哥发生7.1级地震", "自然灾害"),
    ("2017-03-15", "美联储再次加息", "经济事件"),
    ("2017-12-18", "比特币价格接近2万美元，达到历史高点", "经济事件"),
    ("2017-01-20", "特朗普就任美国总统，推行“美国优先”政策", "地缘政治事件"),
    ("2017-06-05", "沙特等国与卡塔尔断交", "地缘政治事件"),
    ("2017-10-01", "加泰罗尼亚独立公投", "地缘政治事件"),
    ("2017-12-06", "特朗普政府承认耶路撒冷为以色列首都", "地缘政治事件"),
    ("2017-06-05", "苹果公司发布HomePod，进入智能音箱市场", "科技进展"),
    ("2017-11-03", "特斯拉发布Semi电动卡车和新款Roadster", "科技进展"),
    ("2017-10-05", "#MeToo运动兴起", "社会文化事件"),
]

events_2018 = [
    ("2018-09-28", "印尼苏拉威西岛地震及海啸", "自然灾害"),
    ("2018-11-08", "美国加州坎普野火，造成严重伤亡和破坏", "自然灾害"),
    ("2018-03-22", "中美贸易战升级，美国对中国商品加征关税", "经济事件"),
    ("2018-06-12", "金特会首次在新加坡举行", "地缘政治事件"),
    ("2018-07-06", "美国退出伊核协议", "地缘政治事件"),
    ("2018-10-02", "卡舒吉遇害案引发国际关注", "地缘政治事件"),
    ("2018-02-06", "SpaceX猎鹰重型火箭首次成功发射", "科技进展"),
    ("2018-10-30", "苹果公司发布新款iPad Pro，采用Face ID", "科技进展"),
    ("2018-06-14", "俄罗斯世界杯开幕", "社会文化事件"),
]

events_2019 = [
    ("2019-03-14", "飓风伊代袭击莫桑比克、津巴布韦和马拉维，造成数百人死亡", "自然灾害"),
    ("2019-04-15", "法国巴黎圣母院大火", "自然灾害"),
    ("2019-09-01", "飓风多里安袭击巴哈马，造成严重破坏", "自然灾害"),
    ("2019-09-17", "亚马逊雨林火灾引发全球关注", "自然灾害"),
    ("2019-05-15", "美国将华为列入实体清单", "经济事件"),
    ("2019-12-13", "中美达成第一阶段贸易协议", "经济事件"),
    ("2019-01-01", "巴西总统博索纳罗就职", "地缘政治事件"),
    ("2019-03-29", "英国脱欧延期", "地缘政治事件"),
    ("2019-06-09", "香港“反修例”运动爆发", "地缘政治事件"),
    ("2019-10-27", "巴格达迪在叙利亚被击毙", "地缘政治事件"),
    ("2019-02-20", "三星发布首款折叠屏手机Galaxy Fold", "科技进展"),
    ("2019-09-10", "苹果公司发布iPhone 11系列", "科技进展"),
    ("2019-03-15", "新西兰基督城清真寺枪击案", "社会文化事件"),
]

events_2020 = [
    ("2020-01-01", "澳大利亚森林大火持续数月，造成巨大生态灾难", "自然灾害"),
    ("2020-07-27", "中国长江流域发生特大洪灾", "自然灾害"),
    ("2020-08-04", "黎巴嫩贝鲁特港口爆炸", "自然灾害"),
    ("2020-01-23", "中国武汉封城，新冠疫情全球蔓延", "经济事件"),
    ("2020-03-09", "“黑色星期一”，全球股市暴跌，原油价格战爆发", "经济事件"),
    ("2020-03-27", "美国通过2万亿美元经济刺激法案", "经济事件"),
    ("2020-01-03", "伊朗高级将领苏莱曼尼被美军击毙，美伊关系紧张", "地缘政治事件"),
    ("2020-01-31", "英国正式脱离欧盟", "地缘政治事件"),
    ("2020-11-03", "美国总统大选，乔·拜登胜出", "地缘政治事件"),
    ("2020-06-22", "苹果公司宣布Mac电脑将改用自研芯片", "科技进展"),
    ("2020-11-10", "索尼发布PlayStation 5，微软发布Xbox Series X/S", "科技进展"),
    ("2020-05-25", "乔治·弗洛伊德事件引发全球“黑人的命也是命”抗议活动", "社会文化事件"),
]

events_2021 = [
    ("2021-02-13", "美国得州遭遇冬季风暴和停电", "自然灾害"),
    ("2021-07-20", "德国和比利时遭遇特大洪灾", "自然灾害"),
    ("2021-08-14", "海地发生7.2级地震", "自然灾害"),
    ("2021-12-10", "美国中西部遭遇龙卷风群袭击", "自然灾害"),
    ("2021-01-01", "RCEP正式签署", "经济事件"),
    ("2021-03-23", "长荣货轮“长赐号”堵塞苏伊士运河", "经济事件"),
    ("2021-10-01", "全球能源危机加剧", "经济事件"),
    ("2021-01-06", "美国国会山骚乱事件", "地缘政治事件"),
    ("2021-08-15", "塔利班重掌阿富汗政权", "地缘政治事件"),
    ("2021-09-15", "AUKUS安全协议宣布", "地缘政治事件"),
    ("2021-02-18", "毅力号火星车成功着陆火星", "科技进展"),
    ("2021-10-28", "Facebook更名为Meta，押注元宇宙", "科技进展"),
    ("2021-07-23", "东京奥运会开幕（因疫情延期）", "社会文化事件"),
]

events_2022 = [
    ("2022-03-01", "南非德班及周边地区遭遇特大洪灾", "自然灾害"),
    ("2022-06-01", "巴基斯坦遭受特大洪灾", "自然灾害"),
    ("2022-07-01", "欧洲多国遭遇极端高温和干旱", "自然灾害"),
    ("2022-09-18", "飓风伊恩袭击美国佛罗里达州", "自然灾害"),
    ("2022-02-24", "俄罗斯入侵乌克兰，国际油价、天然气和粮食价格飙升", "经济事件"),
    ("2022-03-16", "美联储首次加息，全球进入加息周期", "经济事件"),
    ("2022-11-30", "ChatGPT发布", "经济事件"),
    ("2022-02-04", "北京冬奥会开幕", "地缘政治事件"),
    ("2022-09-08", "英国女王伊丽莎白二世逝世", "地缘政治事件"),
    ("2022-10-27", "新一届中共中央政治局常委班子产生", "地缘政治事件"),
    ("2022-07-12", "詹姆斯·韦布空间望远镜发布首批全彩图像", "科技进展"),
    ("2022-10-27", "埃隆·马斯克完成对Twitter的收购", "科技进展"),
    ("2022-11-20", "卡塔尔世界杯开幕", "社会文化事件"),
]

events_2023 = [
    ("2023-02-06", "土耳其-叙利亚大地震，造成超5万人死亡", "自然灾害"),
    ("2023-07-01", "北半球多地遭遇极端高温，创纪录", "自然灾害"),
    ("2023-09-08", "摩洛哥发生6.8级地震", "自然灾害"),
    ("2023-10-07", "阿富汗西部多次发生强震", "自然灾害"),
    ("2023-03-10", "硅谷银行倒闭，引发美国银行危机", "经济事件"),
    ("2023-06-15", "美联储暂停加息，但表示未来可能继续加息", "经济事件"),
    ("2023-11-15", "APEC峰会在旧金山举行，中美关系出现缓和迹象", "经济事件"),
    ("2023-01-01", "芬兰加入北约", "地缘政治事件"),
    ("2023-04-04", "法国爆发大规模退休制度改革抗议", "地缘政治事件"),
    ("2023-10-07", "哈马斯袭击以色列，新一轮巴以冲突爆发", "地缘政治事件"),
    ("2023-11-01", "英美等国签署《布莱切利宣言》，聚焦AI安全", "地缘政治事件"),
    ("2023-03-14", "OpenAI发布GPT-4", "科技进展"),
    ("2023-06-05", "苹果公司发布Vision Pro头显", "科技进展"),
    ("2023-05-06", "查尔斯三世加冕典礼", "社会文化事件"),
]

events_2024 = [
    ("2024-01-01", "日本能登半岛发生7.6级地震", "自然灾害"),
    ("2024-04-03", "中国台湾花莲地区发生7.3级地震", "自然灾害"),
    ("2024-05-01", "印度和孟加拉国遭遇极端高温和洪涝灾害", "自然灾害"),
    ("2024-05-26", "德国南部遭遇特大洪灾", "自然灾害"),
    ("2024-01-01", "金砖国家扩容，沙特、埃及、阿联酋、伊朗、埃塞俄比亚正式加入", "经济事件"),
    ("2024-06-06", "欧洲央行首次降息，全球央行政策分化加剧", "经济事件"),
    ("2024-07-01", "中国和澳大利亚贸易关系进一步改善", "经济事件"),
    ("2024-01-13", "中国台湾地区领导人选举，民进党赖清德胜选", "地缘政治事件"),
    ("2024-02-17", "慕尼黑安全会议召开，俄乌冲突及全球安全问题成焦点", "地缘政治事件"),
    ("2024-05-19", "伊朗总统莱希坠机身亡", "地缘政治事件"),
    ("2024-06-06", "印度大选结果公布，莫迪再次连任总理", "地缘政治事件"),
    ("2024-02-15", "Google发布Gemini 1.5 Pro", "科技进展"),
    ("2024-05-07", "OpenAI发布GPT-4o", "科技进展"),
    ("2024-07-26", "巴黎奥运会开幕", "社会文化事件"),
]

events_2025 = [
    ['2025-01-05', '巴西热带雨林爆发大规模森林火灾，造成383人死亡', '自然灾害'],
    ['2025-03-01', '非洲多国遭遇暴雨洪涝和山体滑坡，至少1180人死亡', '自然灾害'],
    ['2025-03-18', '联合国气候报告预警全球平均气温逼近1.5°C临界点', '环境事件'],
    ['2025-04-11', '阿富汗和巴基斯坦遭遇极端暴雨及洪涝灾害，至少725人死亡', '自然灾害'],
    ['2025-05-24', '巴布亚新几内亚发生大规模山体滑坡，至少670人死亡', '自然灾害'],
    ['2025-06-14', '沙特阿拉伯朝觐期间遭遇极端高温，至少1301人死亡', '自然灾害'],
    ['2025-07-01', '澳大利亚悉尼连续12天气温超45°C，创南半球城市高温纪录', '自然灾害'],
    ['2025-01-01', '沙特、埃及、阿联酋、伊朗、埃塞俄比亚正式加入金砖国家', '地缘政治事件'],
    ['2025-01-20', '特朗普正式就任美国总统', '政治事件'],
    ['2025-02-18', '俄美两国代表团在沙特利雅得举行高级别会谈', '地缘政治事件'],
    ['2025-03-10', '中东地区多国签署能源合作协议，加强区域一体化', '地缘政治事件'],
    ['2025-04-05', '朝鲜宣布成功进行新型洲际导弹试射', '地缘政治事件'],
    ['2025-05-01', '欧盟发布新的数字服务法案，引发科技巨头争议', '地缘政治事件'],
    ['2025-06-01', '全球贸易组织启动新一轮多边贸易谈判', '地缘政治事件'],
    ['2025-01-15', '中国国家统计局公布2024年GDP增长7.2%', '经济事件'],
    ['2025-02-01', '全球主要央行协调降息，以应对经济下行压力', '经济事件'],
    ['2025-03-05', '欧盟宣布对部分进口商品征收碳边境调节税', '经济事件'],
    ['2025-04-01', '全球主要经济体签署数字经济合作框架协议', '经济事件'],
    ['2025-05-10', '国际能源署发布报告，预测全球石油需求将在2025年达到峰值', '经济事件'],
    ['2025-06-20', '世界银行发布全球经济展望报告，预测全球经济增长放缓', '经济事件'],
    ['2025-01-25', '谷歌发布全新量子计算机原型，性能大幅提升', '科技进展'],
    ['2025-03-20', '全球首例人脑芯片植入手术成功，实现意念控制外部设备', '科技进展'],
    ['2025-04-15', '马斯克宣布Neuralink完成首个商用脑机接口产品', '科技进展'],
    ['2025-05-05', '中国成功发射载人登月飞船，计划2027年登月', '科技进展'],
    ['2025-06-10', 'AI通用大模型在多项智力测试中超越人类平均水平', '科技进展'],
    ['2025-01-10', '联合国发布年度人口报告，全球人口突破85亿', '社会文化事件'],
    ['2025-02-20', '全球首部AI生成电影在柏林电影节获奖', '社会文化事件'],
    ['2025-04-20', '全球青年气候行动峰会在纽约举行', '社会文化事件'],
    ['2025-05-15', '世界卫生组织发布全球健康报告，强调心理健康问题日益突出', '社会文化事件'],
    ['2025-06-25', '国际足球联合会宣布2030年世界杯将在非洲举办', '社会文化事件']
]

# Combine all events into a single list
all_events = []
all_events.extend(events_2010)
all_events.extend(events_2011)
all_events.extend(events_2012)
all_events.extend(events_2013)
all_events.extend(events_2014)
all_events.extend(events_2015)
all_events.extend(events_2016)
all_events.extend(events_2017)
all_events.extend(events_2018)
all_events.extend(events_2019)
all_events.extend(events_2020)
all_events.extend(events_2021)
all_events.extend(events_2022)
all_events.extend(events_2023)
all_events.extend(events_2024)
all_events.extend(events_2025)

df_events = pd.DataFrame(all_events, columns=['Date', 'Description', 'Category'])
df_events['Date'] = pd.to_datetime(df_events['Date'])

# Map Chinese categories to English for plotting and analysis
category_mapping = {
    '自然灾害': 'Natural Disaster',
    '事故灾难': 'Accident/Disaster',
    '经济事件': 'Economic Event',
    '地缘政治事件': 'Geopolitical Event',
    '政治事件': 'Political Event',
    '科技进展': 'Technological Advancement',
    '社会文化事件': 'Social/Cultural Event',
    '环境事件': 'Environmental Event'
}
df_events['Category_EN'] = df_events['Category'].map(category_mapping)

print("事件数据已加载并分类。")

# --- 2. Fetch Financial Data ---
start_date = '2010-01-01'
end_date = '2025-12-31' # Fetch data up to the end of 2025

financial_data = {}

# Fetch SSE Composite Index (上证指数) Data using Tushare
print("正在从Tushare获取上证指数数据...")
try:
    df_sse = pro.index_daily(ts_code='000001.SH', start_date=start_date.replace('-', ''), end_date=end_date.replace('-', ''))
    if not df_sse.empty:
        df_sse['trade_date'] = pd.to_datetime(df_sse['trade_date'])
        df_sse = df_sse.sort_values(by='trade_date').set_index('trade_date')
        df_sse = df_sse[['close']] # Keep only the close price
        financial_data['上证指数'] = df_sse
        print(f"成功获取上证指数数据，从 {df_sse.index.min().strftime('%Y-%m-%d')} 到 {df_sse.index.max().strftime('%Y-%m-%d')}.")
    else:
        print("Tushare返回的上证指数数据为空。可能是API限制或指定范围内无数据。")
except Exception as e:
    print(f"获取上证指数数据时发生错误: {e}")
    print("请确保您的Tushare token正确且拥有足够的访问权限。")

# Fetch Nasdaq Composite Index (^IXIC) Data using yfinance
print("正在从yfinance获取纳斯达克指数数据...")
try:
    df_nasdaq = yf.download('^IXIC', start=start_date, end=end_date)
    if not df_nasdaq.empty:
        df_nasdaq = df_nasdaq[['Close']].rename(columns={'Close': 'close'})
        financial_data['纳斯达克指数'] = df_nasdaq
        print(f"成功获取纳斯达克指数数据，从 {df_nasdaq.index.min().strftime('%Y-%m-%d')} 到 {df_nasdaq.index.max().strftime('%Y-%m-%d')}.")
    else:
        print("yfinance返回的纳斯达克指数数据为空。")
except Exception as e:
    print(f"获取纳斯达克指数数据时发生错误: {e}")

# Fetch Dow Jones Industrial Average (^DJI) Data using yfinance
print("正在从yfinance获取道琼斯指数数据...")
try:
    df_dji = yf.download('^DJI', start=start_date, end=end_date)
    if not df_dji.empty:
        df_dji = df_dji[['Close']].rename(columns={'Close': 'close'})
        financial_data['道琼斯指数'] = df_dji
        print(f"成功获取道琼斯指数数据，从 {df_dji.index.min().strftime('%Y-%m-%d')} 到 {df_dji.index.max().strftime('%Y-%m-%d')}.")
    else:
        print("yfinance返回的道琼斯指数数据为空。")
except Exception as e:
    print(f"获取道琼斯指数数据时发生错误: {e}")

# Fetch International Gold Price (GLD ETF) Data using yfinance
print("正在从yfinance获取国际金价 (GLD ETF) 数据...")
try:
    df_gld = yf.download('GLD', start=start_date, end=end_date)
    if not df_gld.empty:
        df_gld = df_gld[['Close']].rename(columns={'Close': 'close'})
        financial_data['国际金价 (GLD)'] = df_gld
        print(f"成功获取国际金价 (GLD) 数据，从 {df_gld.index.min().strftime('%Y-%m-%d')} 到 {df_gld.index.max().strftime('%Y-%m-%d')}.")
    else:
        print("yfinance返回的国际金价 (GLD) 数据为空。")
except Exception as e:
    print(f"获取国际金价 (GLD) 数据时发生错误: {e}")


# --- 3. Analyze Event Impact on Indices ---
# This section calculates the percentage change in each index around each event.
# We define an observation window of 5 trading days after the event.

event_impacts = []
for instrument_name, df_instrument in financial_data.items():
    if not df_instrument.empty:
        trading_dates = df_instrument.index.sort_values()

        for index, event in df_events.iterrows():
            event_date = event['Date']
            event_category = event['Category_EN']
            event_description = event['Description']

            # Find the index of the first trading day on or after the event date
            start_date_candidate_iloc = trading_dates.searchsorted(event_date)

            # Ensure there's a starting trading day
            if start_date_candidate_iloc < len(trading_dates):
                start_price_date = trading_dates[start_date_candidate_iloc]
                start_price = df_instrument.loc[start_price_date]['close']

                # Find the trading day 5 days after the event date
                end_date_iloc = start_date_candidate_iloc + 5
                if end_date_iloc < len(trading_dates):
                    end_price_date = trading_dates[end_date_iloc]
                    end_price = df_instrument.loc[end_price_date]['close']

                    percentage_change = ((end_price - start_price) / start_price) * 100
                    event_impacts.append({
                        '事件日期': event_date,
                        '描述': event_description,
                        '类别': event_category,
                        '金融工具': instrument_name, # Add instrument name
                        '起始价格日期': start_price_date,
                        '结束价格日期': end_price_date,
                        '起始价格': start_price,
                        '结束价格': end_price,
                        '百分比变化': percentage_change
                    })

df_impacts = pd.DataFrame(event_impacts)

# Group by category and instrument and calculate average impact
avg_impact_by_category_instrument = pd.DataFrame()
if not df_impacts.empty:
    avg_impact_by_category_instrument = df_impacts.groupby(['类别', '金融工具'])['百分比变化'].mean().reset_index()
    print("\n按事件类别和金融工具划分的5日事件后平均百分比变化:\n")
    # Pivot for better readability in console output if preferred
    pivot_table = avg_impact_by_category_instrument.pivot(index='类别', columns='金融工具', values='百分比变化')
    print(pivot_table.to_string())
else:
    print("\n未计算出事件影响。这通常是因为无法获取金融数据或事件过于近期。")

# --- 4. Visualization using Plotly ---

# Plot 1: Financial Indices Performance with Major Global Events Markers
fig_multi_index = go.Figure()

# Define colors for categories
category_colors = {
    'Natural Disaster': px.colors.qualitative.Plotly[0],
    'Accident/Disaster': px.colors.qualitative.Plotly[1],
    'Economic Event': px.colors.qualitative.Plotly[2],
    'Geopolitical Event': px.colors.qualitative.Plotly[3],
    'Political Event': px.colors.qualitative.Plotly[4],
    'Technological Advancement': px.colors.qualitative.Plotly[5],
    'Social/Cultural Event': px.colors.qualitative.Plotly[6],
    'Environmental Event': px.colors.qualitative.Plotly[7]
}
chinese_category_names = {
    'Natural Disaster': '自然灾害',
    'Accident/Disaster': '事故灾难',
    'Economic Event': '经济事件',
    'Geopolitical Event': '地缘政治事件',
    'Political Event': '政治事件',
    'Technological Advancement': '科技进展',
    'Social/Cultural Event': '社会文化事件',
    'Environmental Event': '环境事件'
}


# Add traces for each financial instrument (modified to use normalized data)
# Normalize data to start at 100 for better visual comparison of trends
normalized_financial_data = {}
for instrument_name, df_instrument in financial_data.items():
    if not df_instrument.empty:
        # Calculate percentage change from the first available close price
        # Using .iloc[0] for robustness against non-contiguous indices
        first_price = df_instrument['close'].iloc[0]
        df_instrument['normalized_close'] = (df_instrument['close'] / first_price) * 100
        normalized_financial_data[instrument_name] = df_instrument

# Add traces for each financial instrument using normalized data
for instrument_name, df_instrument in normalized_financial_data.items():
    if not df_instrument.empty:
        fig_multi_index.add_trace(go.Scatter(x=df_instrument.index, y=df_instrument['normalized_close'], mode='lines', name=instrument_name,
                                             line=dict(width=2)))

# Add event markers
if not df_impacts.empty and '上证指数' in normalized_financial_data: # Ensure SSE data is available for marker placement
    sse_normalized_df = normalized_financial_data['上证指数']
    legend_added_categories = set() # To control legend entries for event categories

    for idx, event in df_events.iterrows():
        event_date = event['Date']
        event_category_en = event['Category_EN']
        event_description = event['Description']

        # Get the normalized SSE price at the event date for the marker's y-position
        # Find the actual trading date closest to or on the event_date
        trading_dates_sse = sse_normalized_df.index.sort_values()

        # Check if event_date is within trading_dates_sse's range
        if event_date >= trading_dates_sse.min() and event_date <= trading_dates_sse.max():
            # Get the actual trading date closest to or on the event_date
            closest_trade_date_idx = trading_dates_sse.searchsorted(event_date)

            # Handle edge case where searchsorted might return len(trading_dates_sse)
            if closest_trade_date_idx == len(trading_dates_sse):
                continue # No trading date on or after event_date, skip this event

            # If the event_date itself is not a trading date, use the next available trading date
            closest_trade_date = trading_dates_sse[closest_trade_date_idx] if (closest_trade_date_idx < len(trading_dates_sse) and trading_dates_sse[closest_trade_date_idx] == event_date) else trading_dates_sse[closest_trade_date_idx]


            # Set marker Y-position to a fixed value (e.g., 90) on the normalized scale
            # This places the markers near the bottom of the normalized chart, on the "x-axis"
            event_marker_y_position = 10

            # Prepare hover text for this specific event across all instruments
            single_event_impact_df = df_impacts[df_impacts['描述'] == event_description]
            detailed_impact_text = []
            # Sort for consistent hover text order
            for _, impact_row in single_event_impact_df.sort_values(by='金融工具').iterrows():
                instrument = impact_row['金融工具']

                # Ensure impact_val is a scalar by using .item() if it's a Series, otherwise use as is
                impact_val_raw = impact_row['百分比变化']
                impact_val = impact_val_raw.item() if isinstance(impact_val_raw, pd.Series) else impact_val_raw

                impact_str = f"{impact_val:.2f}" if pd.notna(impact_val) else "N/A"
                detailed_impact_text.append(f"{instrument} 影响: {impact_str}%")

            full_hover_text = (
                f"事件: {event_description}<br>"
                f"类别: {event['Category']}<br>" # Corrected column name here
                f"日期: {event_date.strftime('%Y-%m-%d')}<br>" +
                "<br>".join(detailed_impact_text)
            )

            # Determine if this category's legend entry should be shown
            show_this_legend = False
            if event_category_en not in legend_added_categories:
                show_this_legend = True
                legend_added_categories.add(event_category_en)

            fig_multi_index.add_trace(go.Scatter(
                x=[event_date], # Plot marker at original event date
                y=[event_marker_y_position], # Use fixed Y-coordinate for markers
                mode='markers',
                name=f'事件类别: {chinese_category_names.get(event_category_en, event_category_en)}', # Use Chinese category name for legend
                marker=dict(
                    symbol='star',
                    size=12,
                    color=category_colors.get(event_category_en, 'grey'), # Use predefined color for category
                    line=dict(width=1, color='DarkSlateGrey')
                ),
                text=[full_hover_text],
                hoverinfo='text',
                showlegend=show_this_legend, # Control legend visibility
                legendgroup=event_category_en # Group markers of the same category in the legend
            ))

fig_multi_index.update_layout(
    title_text='主要全球事件与金融工具归一化表现 (2010-2025)', # 修正标题以反映归一化
    xaxis_title='日期', # 中文X轴标签
    yaxis_title='归一化指数值/价格 (基准=100)', # 中文Y轴标签以反映归一化
    hovermode='x unified', # Hover mode remains x unified for main lines
    legend_title_text='图例', # 中文图例标题
    template='plotly_white',
    height=700, # Increased height for more data
    xaxis_rangeselector_buttons=list([
        dict(count=1, label="1个月", step="month", stepmode="backward"),
        dict(count=6, label="6个月", step="month", stepmode="backward"),
        dict(count=1, label="1年", step="year", stepmode="backward"),
        dict(step="all", label="全部")
    ]),
    xaxis_rangeslider_visible=True,
)
fig_multi_index.show()


# Plot 2: Average Impact by Event Category for Each Instrument
if not avg_impact_by_category_instrument.empty:
    fig_bar_multi = px.bar(
        avg_impact_by_category_instrument,
        x='类别',
        y='百分比变化',
        color='金融工具', # Use instrument as color for grouping
        barmode='group', # Group bars by category
        title='按事件类别和金融工具划分的平均指数变化 (5日事件后)', # 中文标题
        labels={'类别': '事件类别', '百分比变化': '平均百分比变化 (%)', '金融工具': '金融工具'}, # 中文标签
        color_continuous_scale=px.colors.sequential.RdBu,
        template='plotly_white'
    )
    fig_bar_multi.update_layout(
        xaxis_title='事件类别', # 中文X轴标签
        yaxis_title='平均百分比变化 (%)', # 中文Y轴标签
        xaxis_tickangle=-45,
        height=600 # Increased height for more data
    )
    fig_bar_multi.show()
else:
    print("\n未计算出事件影响，跳过平均影响柱状图。")
```

```
事件数据已加载并分类。
正在从Tushare获取上证指数数据...
成功获取上证指数数据，从 2010-01-04 到 2025-06-04.
正在从yfinance获取纳斯达克指数数据...
[*********************100%***********************]  1 of 1 completed
成功获取纳斯达克指数数据，从 2010-01-04 到 2025-06-04.
正在从yfinance获取道琼斯指数数据...
[*********************100%***********************]  1 of 1 completed
成功获取道琼斯指数数据，从 2010-01-04 到 2025-06-04.
正在从yfinance获取国际金价 (GLD ETF) 数据...
[*********************100%***********************]  1 of 1 completed
成功获取国际金价 (GLD) 数据，从 2010-01-04 到 2025-06-04.
```





```
按事件类别和金融工具划分的5日事件后平均百分比变化:

金融工具                          上证指数  国际金价 (GLD)    纳斯达克指数      道琼斯指数
类别
Accident/Disaster         -1.585208   1.942547 -2.323543  -1.99817
Economic Event            -0.628204    -0.2402 -0.207979  -0.40546
Environmental Event       -1.743067  -0.532215  4.386055  2.419817
Geopolitical Event         0.567729  -0.085553  0.901015  0.601778
Natural Disaster          -0.410094   0.247901 -0.452765 -0.104475
Political Event            0.191812   0.809856 -0.117375  1.872863
Social/Cultural Event      0.079481   0.306388   0.14235  0.059713
Technological Advancement  0.406746   0.268876  0.274548  0.352043
```